

Gaussian Beam Simulation, Optimization and Analysis

Himanshu Sharma

Contents

1	Initial Code: Direct Summation Method	3
1.1	MATLAB Implementation	3
2	Mathematics of the Initial Code	4
3	Errors in the Initial Code	4
3.1	Error 1: Incorrect Evanescent Wave Handling	4
3.2	Error 2: Missing Integration Scale Factor	5
3.3	Mathematical Corrections	5
4	Optimization 1: Vectorization (Matrix Multiplication)	5
4.1	Mathematical Derivation	5
4.2	Mock Code	6
5	Optimization 2: Angular Spectrum Method (ASM)	6
5.1	Mathematical Derivation (ASM)	7
5.2	MATLAB Implementation (ASM)	7
6	Analysis of Optimization 2 (ASM)	8
6.1	Problem: Spatial Aliasing (Boundary Ripples)	8
6.2	Solution: Expanding the Simulation Window	9
7	Comparative Analysis and Final Corrections	9
7.1	Parameter Matching for Comparison	9
7.2	Problem 1: Computational Inefficiency (Memory Error)	10
7.3	Solution 1: Reducing Grid Size	10
7.4	Problem 2: Normalization and Scaling Error	10
7.5	Mathematical Explanation of Scaling Error	10
7.6	Final Corrected Code (Vectorized Method)	10
8	Summary of Results for Basic Simulation Optimization	12
9	Plotting the Poynting Vector	12
9.1	Introduction	12
9.2	Background Theory	12
9.2.1	The Poynting Vector	12
9.2.2	Paraxial Approximation	13
9.3	Mathematical Formulation	13
9.3.1	From Maxwell's Equations to the H-Field	13
9.3.2	The Poynting Vector Calculation	13
9.3.3	Numerical Derivatives	13
9.4	Plots and Results	14

10 Plotting the beam at an interface	15
10.1 Introduction	15
10.2 Mathematical Formulation	15
10.2.1 Decomposing the Tilted Beam	15
10.2.2 Fresnel Equations	15
10.2.3 Superposition of Fields	16
10.3 Plots	16
10.4 The Current Model (Circular Projection)	17
10.4.1 Mathematical Formulation	17
10.4.2 Physical Interpretation	17
10.5 The Corrected Model (Elliptical Projection)	18
10.5.1 Mathematical Derivation: The Rotated Beam Model	18
10.5.2 Physical Interpretation	20
10.6 Comparative Analysis of Simulation Results	20
10.6.1 Case 4: Brewster Angle (s-pol) - Reflected Beam	20
10.7 Case 7: Total Internal Reflection (TIR) - Evanescent Decay	21

1 Initial Code: Direct Summation Method

We begin with the provided MATLAB function. This code attempts to simulate a Gaussian beam by directly summing a discrete set of plane waves.

1.1 MATLAB Implementation

```
1 function [X, Y, E_tmp] = Gaussian_Beam(z)
2
3 lambda = 1; k = 2*pi/lambda; sigma = 0.1*k;
4
5 kx = linspace(-0.3, 0.3, 50)*k;
6 ky = linspace(-0.3, 0.3, 50)*k;
7 [Kx, Ky] = meshgrid(kx, ky);
8
9 Kz = (k^2 - Kx.^2 - Ky.^2).^0.5;
10
11 A = exp(-(Kx.^2 + Ky.^2)/(2*sigma^2));
12
13 x = linspace(-3, 3, 30)*lambda;
14 y = linspace(-3, 3, 30)*lambda;
15 [X, Y] = meshgrid(x, y);
16
17 E = zeros(size(X));
18
19 for i1 = 1 : size(Kz,1)
20     for i2 = 1 : size(Kz,1)
21         Atmp = A(i1,i2)*exp(-1i*Kz(i1,i2)*z*lambda/450);
22         E = E + Atmp*exp(-1i*(Kx(i1,i2).*X + Ky(i1,i2).*Y));
23     end
24 end
25
26 E_tmp = real(E);
27
28 end
```

Listing 1: Initial Gaussian beam simulation using for-loops

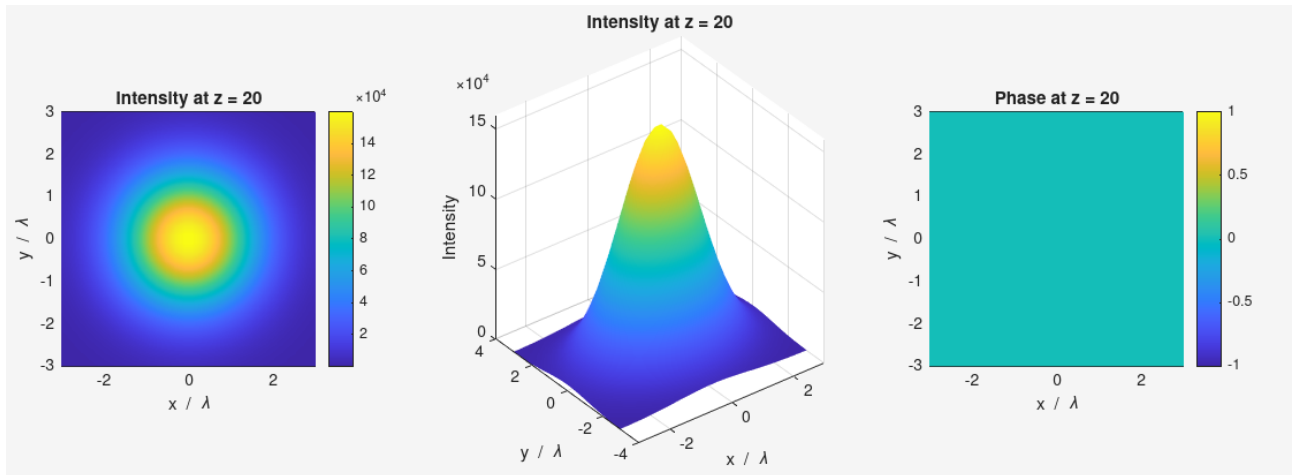


Figure 1: Intensity and Phase plots for the initial code

2 Mathematics of the Initial Code

The code is a numerical implementation of the **plane wave superposition principle**, which is a fundamental solution to the scalar Helmholtz equation for a monochromatic field E :

$$(\nabla^2 + k^2)E = 0$$

where $k = 2\pi/\lambda$ is the wavenumber.

A general solution can be represented as a continuous superposition (an integral) of plane waves. The electric field E at any point (x, y, z) is given by:

$$E(x, y, z) = \frac{1}{(2\pi)^2} \iint_{-\infty}^{\infty} A(k_x, k_y) e^{-i(k_x x + k_y y + k_z z)} dk_x dk_y$$

This is an inverse 2D Fourier transform. The term $A(k_x, k_y)$ is the **angular spectrum**, representing the complex amplitude of the plane wave component with wave vector $\mathbf{k} = (k_x, k_y, k_z)$.

To satisfy the Helmholtz equation, the components must be related by the dispersion relation:

$$k_x^2 + k_y^2 + k_z^2 = k^2 \implies k_z = \sqrt{k^2 - k_x^2 - k_y^2}$$

The initial code attempts to discretize this integral:

$$\iint \dots dk_x dk_y \approx \sum_{i1} \sum_{i2} \dots \Delta k_x \Delta k_y$$

- `A = exp(-(Kx.^2 + Ky.^2)/(2*sigma2))`: This defines the angular spectrum $A(k_x, k_y)$ as a Gaussian function. The k -space distribution is also a Gaussian because the Fourier Transform of a Gaussian is also a Gaussian.
- `Atmp = A(i1,i2)*exp(-1i*Kz(i1,i2)*z*...)`: calculates the propagated amplitude. The term $e^{-ik_z z}$ is the **propagation kernel**, which applies the correct phase shift to each plane wave as it travels a distance z .
- `E = E + Atmp*exp(-1i*(Kx*...))`: This is the core summation. Each term `Atmp*...` represents one plane wave $A(k_x, k_y) e^{-ik_z z} \cdot e^{-i(k_x x + k_y y)}$ evaluated at all points (X, Y) on the grid.
- The nested `for` loops perform the discrete summation $\sum_{i1} \sum_{i2}$ over all k_x and k_y components.

The code builds the field $E(x, y, z)$ by adding up $50 \times 50 = 2500$ individual plane waves, each with its own amplitude, direction (k_x, k_y, k_z) , and propagation phase.

3 Errors in the Initial Code

3.1 Error 1: Incorrect Evanescent Wave Handling

In the line `Kz = (k2 - Kx.^2 - Ky.^2).^0.5`, consider large k_x or k_y (high-angle waves) such that $k_x^2 + k_y^2 > k^2$.

- The term $k^2 - k_x^2 - k_y^2$ becomes negative.
- Let $k_x^2 + k_y^2 - k^2 = \alpha^2$, where α is real.
- The code calculates $k_z = \sqrt{-\alpha^2} = i\alpha$.
- The propagation term becomes: $e^{-ik_z z} = e^{-i(i\alpha)z} = e^{\alpha z}$.

This is an **exponentially growing** wave, which is physically non-existent. These components are **evanescent waves** and must decay.

3.2 Error 2: Missing Integration Scale Factor

The code discretizes an integral:

$$E \approx \sum_{i1} \sum_{i2} [A(k_{x,i1}, k_{y,i2}) \dots]$$

However, the proper numerical approximation of a 2D integral is:

$$\iint f(k_x, k_y) dk_x dk_y \approx \sum_{i1} \sum_{i2} f(k_{x,i1}, k_{y,i2}) \Delta k_x \Delta k_y$$

The code is missing the differential area element $\mathbf{dkx} * \mathbf{dky}$. This means the resulting field E has incorrect amplitude and is dependent on the k-space grid sampling (changing from 50 to 100 points would change the final amplitude).

3.3 Mathematical Corrections

1. **Correcting Evanescent Waves:** The physical k_z for evanescent waves should be imaginary with a negative sign: $k_z = -i\alpha = -i\sqrt{k_x^2 + k_y^2 - k^2}$. This leads to the correct exponentially damping propagation:

$$e^{-ik_z z} = e^{-i(-i\alpha)z} = e^{-\alpha z}$$

2. **Correcting Scaling:** The summation must include the differential area element. The loop should be:

```
dkx = kx(2) - kx(1);
dky = ky(2) - ky(1);
E = E + Atmp*... * (dkx * dky);
```

4 Optimization 1: Vectorization (Matrix Multiplication)

The nested `for` have a complexity of $O(N_{kx} \cdot N_{ky} \cdot N_x \cdot N_y)$, or $O(N^4)$ if all grid dimensions are $\sim N$.

We can transform the summation as a single matrix-vector product. We flatten the 2D spatial grid (X, Y) into a 1D vector and the 2D k-space grid (Kx, Ky) into another 1D vector.

4.1 Mathematical Derivation

The corrected discrete sum is:

$$E(x_i, y_i) = \sum_j \sum_l A_z(k_{x,j}, k_{y,l}) e^{-i(k_{x,j}x_i + k_{y,l}y_i)}$$

(Here A_z includes the amplitude A , propagator $e^{-ik_z z}$, and scaling $\Delta k_x \Delta k_y$).

- Let the spatial grid points be indexed by a single index $p = 1, \dots, N_x N_y$.
- Let the k-space grid points be indexed by a single index $q = 1, \dots, N_{kx} N_{ky}$.

The sum becomes:

$$E_p = \sum_{q=1}^{N_{kx} N_{ky}} \left[e^{-i(k_{x,q}x_p + k_{y,q}y_p)} \right] \cdot A_{z,q}$$

This is the definition of a matrix-vector product, $\mathbf{E} = \mathbf{M} \cdot \mathbf{A}_z$:

$$\begin{pmatrix} E_1 \\ E_2 \\ \vdots \\ E_p \end{pmatrix} = \begin{pmatrix} M_{1,1} & M_{1,2} & \dots & M_{1,q} \\ M_{2,1} & M_{2,2} & \dots & M_{2,q} \\ \vdots & \vdots & \ddots & \vdots \\ M_{p,1} & M_{p,2} & \dots & M_{p,q} \end{pmatrix} \begin{pmatrix} A_{z,1} \\ A_{z,2} \\ \vdots \\ A_{z,q} \end{pmatrix}$$

where the matrix element $M_{p,q}$ is the "phase-matching" term:

$$M_{p,q} = e^{-i(k_{x,q}x_p + k_{y,q}y_p)}$$

The line `Phase_Matrix = exp(-1i * (X_v * Kx_v.' + Y_v * Ky_v.'));` constructs this exact matrix M using MATLAB's broadcasting (outer product) capabilities.

4.2 Mock Code

```

1  Kx_v = Kx(:); % 2500x1 vector
2  Ky_v = Ky(:); % 2500x1 vector
3  A_z_v = A_z(:);
4
5  X_v = X(:); % 900x1 vector
6  Y_v = Y(:); % 900x1 vector
7
8  % Create phase-matching matrix M, where M(i,j) = exp(-i*(kx_j*x_i + ky_j*y_i))
9  Phase_Matrix = exp(-1i * (X_v * Kx_v.' + Y_v * Ky_v.'));
10
11 % E_v(i) = SUM_j [ M(i,j) * A_z_v(j) ]
12 E_v = Phase_Matrix * A_z_v;
13
14 E_complex = reshape(E_v, size(X));

```

Listing 2: Mock code for vectorized summation

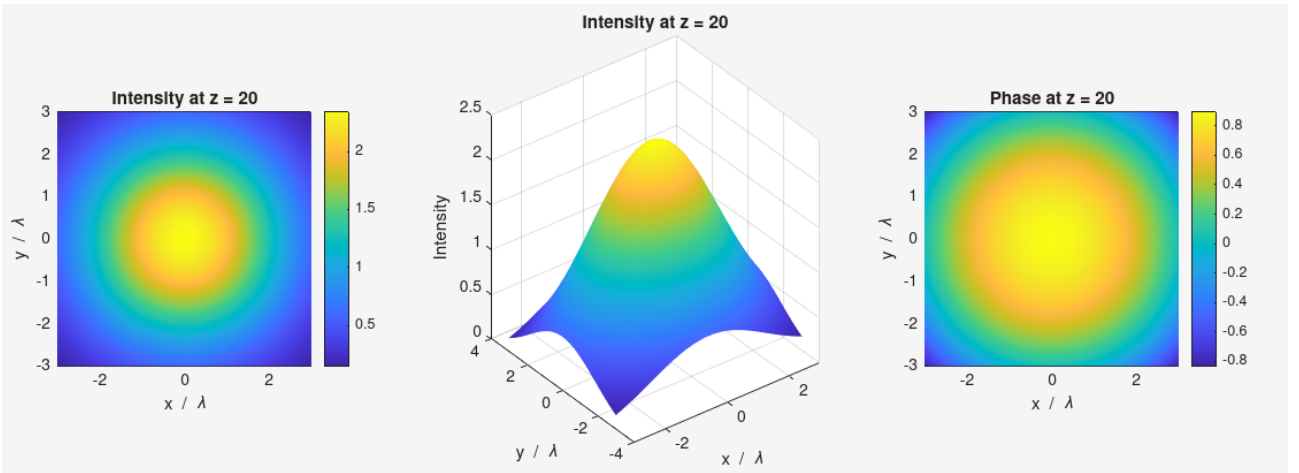


Figure 2: Intensity and Phase Plots for Vectorization-based Optimization

5 Optimization 2: Angular Spectrum Method (ASM)

Optimization 1 is still computationally and memory-intensive. A much faster method, the Angular Spectrum Method (ASM), uses the Fast Fourier Transform (FFT).

5.1 Mathematical Derivation (ASM)

The ASM method is a direct implementation of the physics using the **Convolution Theorem**.

1. We start with the field $E(x, y, z)$. Its 2D Fourier transform (FT) is the angular spectrum $A(k_x, k_y, z)$.

$$A(k_x, k_y, z) = \mathcal{F}\{E(x, y, z)\}$$

2. The solution to the Helmholtz equation in k-space is simple. The spectrum at z is related to the spectrum at $z = 0$ by the propagator $H = e^{-ik_z z}$:

$$A(k_x, k_y, z) = A(k_x, k_y, 0) \cdot H(k_x, k_y, z)$$

This is because propagation in real space (a complex convolution) is equivalent to a simple multiplication in the frequency (k-space) domain.

3. To find the real-space field $E(x, y, z)$, we take the inverse Fourier transform (IFT) of the propagated spectrum:

$$E(x, y, z) = \mathcal{F}^{-1}\{A(k_x, k_y, z)\}$$

This gives a 3-step algorithm, which the code follows exactly:

1. **Step 1:** `A_z0 = fftshift(fft2(E_z0));`

Calculates $A(k_x, k_y, 0) = \mathcal{F}\{E(x, y, 0)\}$. `fft2` is the Fast Fourier Transform, a $O(N^2 \log N)$ algorithm for the Discrete Fourier Transform (DFT). `fftshift` rearranges the output to be human-readable (zero-frequency at the center).

2. **Step 2:** `A_z = A_z0 .* Propagator;`

Performs the k-space multiplication $A(k_x, k_y, z) = A_0 \cdot H$. This is an $O(N^2)$ element-wise operation.

3. **Step 3:** `E_complex = ifft2(ifftshift(A_z));`

Calculates $E(x, y, z) = \mathcal{F}^{-1}\{A(k_x, k_y, z)\}$. `ifftshift` reverses the `fftshift` before the Inverse FFT.

This $O(N^2 \log N)$ method is vastly superior to the $O(N^4)$ direct summation or $O(N^3)$ (for $N_{\text{spatial}} \sim N_k$) matrix multiplication.

5.2 MATLAB Implementation (ASM)

```
1 function [X, Y, E_complex] = Gaussian_Beam_ASM(z_distance)
2     lambda = 1;
3     k = 2*pi/lambda;
4
5     N = 256;           % Number of grid points (must be power of 2 for FFT)
6     L = 10 * lambda;   % Total grid size in x and y
7
8     dx = L / N;
9     x = (-N/2 : N/2 - 1) * dx;
10    y = x;
11    [X, Y] = meshgrid(x, y);
12
13    % We define the initial beam in REAL-SPACE
14    w0 = 1.6 * lambda;
15    E_z0 = exp(-(X.^2 + Y.^2) / (w0^2));
16
17    % Create k-space Grid
18    % FFT links real-space and k-space grids
19    dk = 2*pi/L;
20    kx_v = (-N/2 : N/2 - 1) * dk;
```

```

21 ky_v = kx_v;
22 [Kx, Ky] = meshgrid(kx_v, ky_v);
23
24 % Calculate Kz (The Propagator Kernel)
25 Kz_sq = k^2 - Kx.^2 - Ky.^2;
26 Kz = sqrt(Kz_sq);
27 Kz(Kz_sq < 0) = -1i * sqrt(abs(Kz_sq(Kz_sq < 0)));
28
29 Propagator = exp(-1i * Kz * z_distance);
30
31 % FFT to get Angular Spectrum
32 A_z0 = fftshift(fft2(E_z0));
33
34 % Propagate in k-space
35 A_z = A_z0 .* Propagator;
36
37 % IFFT to get new Real-Space Field
38 E_complex = ifft2(ifftshift(A_z));
39 end

```

Listing 3: Gaussian beam propagation using the Angular Spectrum Method (ASM)

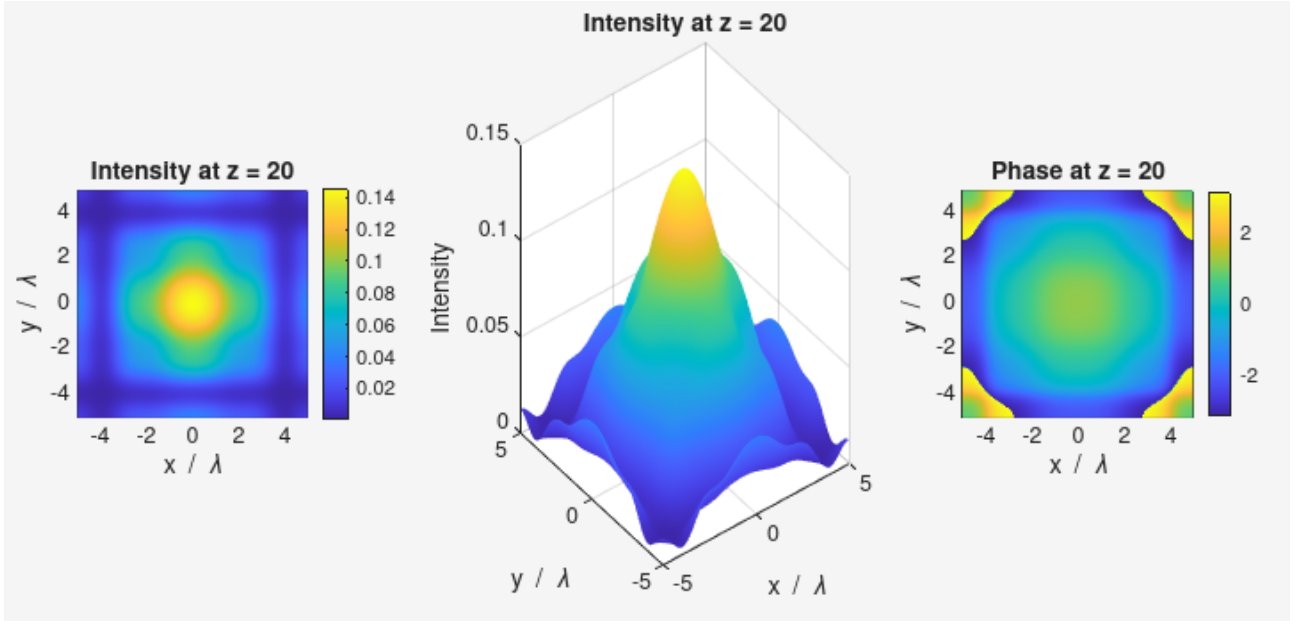


Figure 3: Intensity and Phase Plots for ASM-based Optimization (ripples observed at edges)

6 Analysis of Optimization 2 (ASM)

6.1 Problem: Spatial Aliasing (Boundary Ripples)

When this simulation is run, ripples are often observed at the edges of the simulation window. This is an artifact of the Discrete Fourier Transform (DFT), which inherently assumes the signal is **periodic**.

Mathematically, the DFT assumes $E(x + L, y) = E(x, y)$ and $E(x, y + L) = E(x, y)$.

1. As the beam propagates, it diffracts and expands.
2. The beam's electric field $E(x, y, z)$ may become non-zero at the boundary (e.g., at $x = L/2$).

3. Due to the periodic boundary condition, this non-zero field wraps around and re-enters the simulation box at the opposite side (e.g., at $x = -L/2$).
4. This wrapped field interferes with the main, central part of the beam, causing artificial interference fringes (ripples).

This is a form of spatial aliasing where the expanding beam aliases with its own periodic copy.

6.2 Solution: Expanding the Simulation Window

The solution is to make the box large enough that the beam never (or negligibly) reaches the boundary within the propagation distance. By changing $L = 10 * \text{lambda}$ to $L = 30 * \text{lambda}$, we create a much larger "empty" buffer zone around the beam. The beam can now expand freely without its edges wrapping around and causing interference.

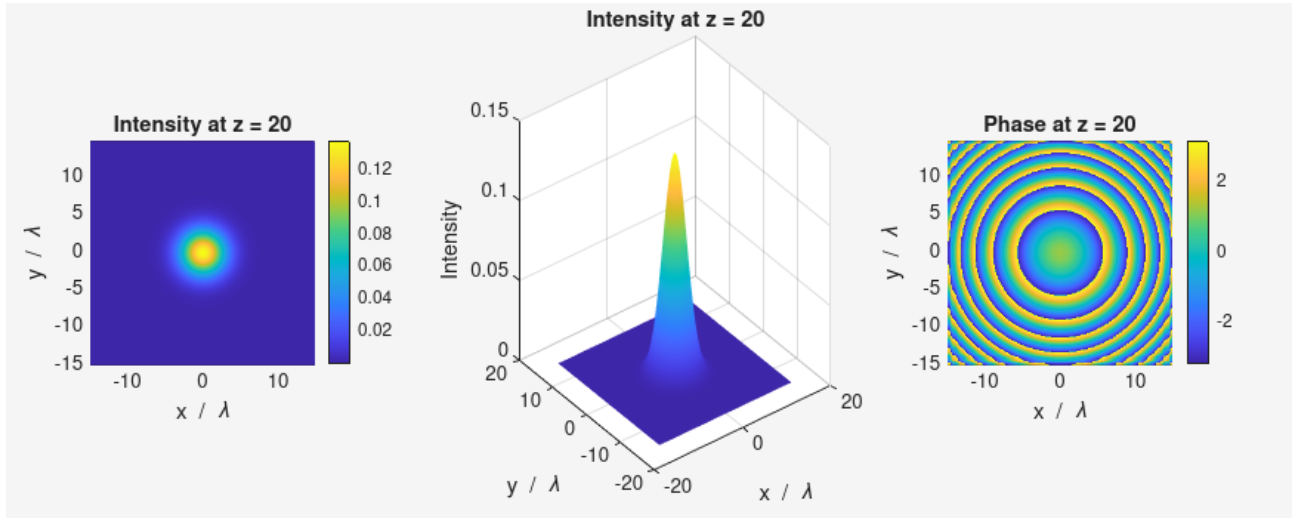


Figure 4: Intensity and Phase Plots for ASM-Based Optimization with an increased simulation window

7 Comparative Analysis and Final Corrections

We now compare Optimization 1 (Vectorized Sum) with Optimization 2 (ASM). For this, we first require both the simulated Gaussian beams to be the same. That is, they must have the same parameters and normalization rules.

7.1 Parameter Matching for Comparison

To compare them, all grid parameters must be identical.

```

1 L = 30 * lambda; % This L is the spatial window
2 N = 256;         % Number of grid points
3 dx = L / N;
4 x = (-N/2 : N/2 - 1) * dx;
5 y = x;
6 [X, Y] = meshgrid(x, y);
7
8 % Define k-space grid consistent with FFT
9 dk = 2*pi/L;
10 kx_v = (-N/2 : N/2 - 1) * dk;
11 ky_v = kx_v;

```

```
12 [Kx, Ky] = meshgrid(kx_v, ky_v);
```

Listing 4: Matching parameters for both methods

7.2 Problem 1: Computational Inefficiency (Memory Error)

With $N = 256$, the total number of grid points is $N^2 = 256^2 = 65,536$. The `Phase_Matrix` from Optimization 1 has dimensions $[(N \cdot N) \times (N \cdot N)]$, or $65,536 \times 65,536$. This matrix requires $(65536)^2 \times 16$ bytes/element ≈ 68.7 GB of RAM. This fails with the error: `Requested 65536x65536 (32.0GB) array exceeds maximum array size preference.` (Note: MATLAB's estimate differs as it may be for double precision, not complex).

7.3 Solution 1: Reducing Grid Size

To make Optimization 1 runnable, we must reduce the grid size for both simulations, e.g., $N = 64$. This makes the `Phase_Matrix` 4096×4096 , which is manageable (~ 268 MB).

7.4 Problem 2: Normalization and Scaling Error

After fixing $N = 64$, we find that the intensity profiles from Optimization 1 and Optimization 2 are wildly different. This is a **normalization and scaling problem**. The direct sum (Opt 1) is incorrectly scaled, while the `fft2/iff2` pair (Opt 2) is internally consistent.

7.5 Mathematical Explanation of Scaling Error

The physical inverse Fourier transform is:

$$E(x, y) = \frac{1}{(2\pi)^2} \iint A(k_x, k_y) e^{-i(k_x x + k_y y)} dk_x dk_y$$

Our discrete approximation (Opt 1) must match this:

$$E(x_i, y_i) \approx \frac{1}{(2\pi)^2} \sum_j \sum_l A(k_{x,j}, k_{y,l}) e^{-i(k_{x,j} x_i + k_{y,l} y_i)} \Delta k_x \Delta k_y$$

Optimization 1 was missing three key factors:

1. The $\frac{1}{(2\pi)^2}$ prefactor.
2. The differential area element $\Delta k_x \Delta k_y$ (or `dkx * dky`) as pointed in the previous sections.
3. The analytic prefactor for $A(k_x, k_y)$: The ASM code starts with a real-space Gaussian $E_0 = \exp(-(r/w_0)^2)$. Its analytic Fourier transform is $A_0 = \frac{\pi w_0^2}{2} \exp(-k^2 w_0^2/4)$. (Note: exact form depends on FT definition). The initial k-space Gaussian in Opt 1, `A = exp(...)`, was just a shape; it lacked the correct amplitude (prefactor) to match the initial E_0 from the ASM code.

7.6 Final Corrected Code (Vectorized Method)

This code corrects the scaling of Optimization 1 to match the physical result of Optimization 2.

```
1 % For Matrix Multiplication based optimization code
2 lambda = 1;
3 k = 2*pi/lambda;
4
5 N = 64; % to avoid memory error
6 L = 30 * lambda;
7 dk = 2*pi/L;
```

```

8 kx_v = (-N/2 : N/2 - 1) * dk;
9 ky_v = kx_v;
10 [Kx, Ky] = meshgrid(kx_v, ky_v);
11
12 sigma = 0.1*k;
13
14 % Add the analytic prefactor to A ---
15 w0 = sqrt(2) / sigma;
16 A_prefactor = pi * w0^2; % From analytic FT of a Gaussian
17 A = A_prefactor * exp(-(Kx.^2 + Ky.^2)/(2*sigma^2));
18 % ---
19
20 Kz_sq = k^2 - Kx.^2 - Ky.^2;
21 Kz = sqrt(Kz_sq);
22 Kz(Kz_sq < 0) = -1i * sqrt(abs(Kz_sq(Kz_sq < 0)));
23 Propagator = exp(-1i * Kz * 0); % z=0 for simplicity
24
25 dx = L / N;
26 x = (-N/2 : N/2 - 1) * dx;
27 y = x;
28 [X, Y] = meshgrid(x, y);
29
30 % Add the integration element dk_x * dk_y
31 dkx = kx_v(2) - kx_v(1);
32 dky = ky_v(2) - ky_v(1);
33 A_z = A .* Propagator * (dkx * dky); % This is now A_true * d(area)
34
35 A_z_v = A_z(:);
36 X_v = X(:);
37 Y_v = Y(:);
38 Phase_Matrix = exp(-1i * (X_v * Kx_v.' + Y_v * Ky_v.'));
39 E_v = Phase_Matrix * A_z_v;
40 E = reshape(E_v, size(X));
41
42 % Add the inverse transform prefactor
43 E = E / (4 * pi^2);
44 % ---
45 end

```

Listing 5: Fully corrected vectorized summation (Opt 1)

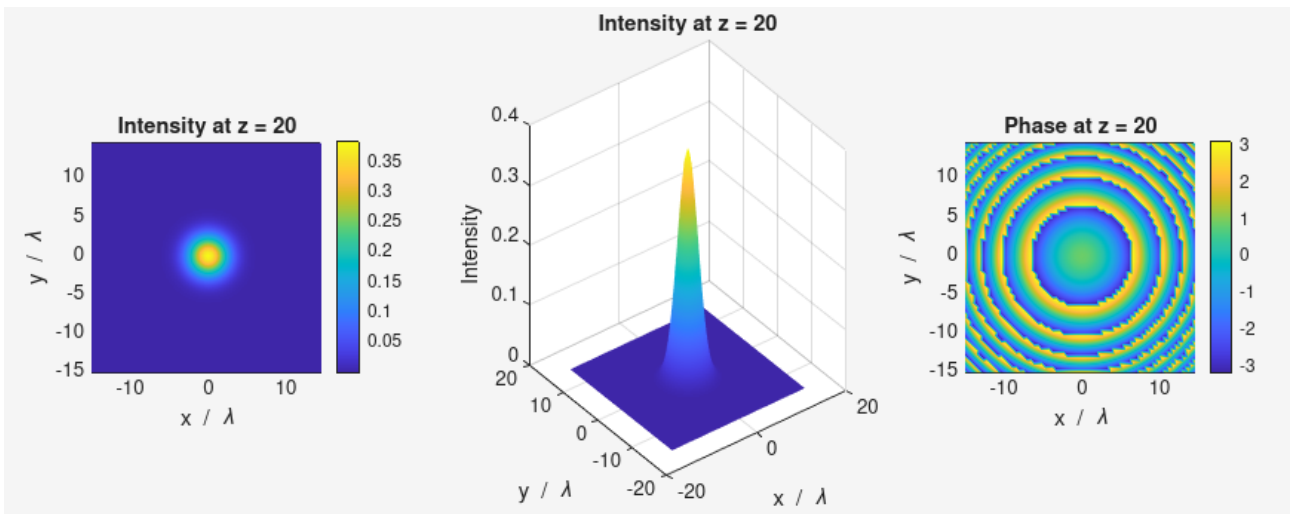


Figure 5: Normalized and Corrected Intensity and Phase Plots for Vectorization-based Optimization

8 Summary of Results for Basic Simulation Optimization

This documentation traced the simulation of a Gaussian beam from a naive, error-filled `for`-loop implementation to a robust, industry-standard method.

1. **Initial Code:** A direct, discrete summation of plane waves. It was functionally $O(N^4)$, physically incorrect (evanescent waves), and numerically unscaled (missing $\Delta k_x \Delta k_y$).
2. **Optimization 1 (Vectorization):** Re-cast the problem as an $O(N^3)$ matrix multiplication. This is more efficient in MATLAB but is extremely memory-intensive, failing for grids as small as $N = 256$.
3. **Optimization 2 (ASM):** Used the Angular Spectrum Method, which leverages the $O(N^2 \log N)$ FFT algorithm. This is the correct and standard approach: it is fast, memory-efficient, and physically sound.
4. **Artifacts and Fixes:** The ASM method's reliance on DFT introduces periodic boundary conditions, causing wrap-around interference. This was solved by expanding the simulation window (L), creating a buffer.
5. **Final Comparison:** To make the Vectorization based Optimization 1 match the Second Optimization based on FFT and IFFT, we had to (a) reduce the grid size to $N = 64$ to avoid memory errors and (b) add all missing physical and numerical scaling factors: the $\frac{1}{(2\pi)^2}$ from the FT definition, the $\Delta k_x \Delta k_y$ from the Riemann sum, and the analytic amplitude prefactor for the $A(k_x, k_y)$ spectrum.

Conclusion: The Angular Spectrum Method (Optimization 2) is the superior method for beam propagation.

9 Plotting the Poynting Vector

9.1 Introduction

In this section, we provide a validation of the beam propagation model. While we often assume a beam travels purely in the z -direction (the paraxial approximation), this is not strictly true. A focused beam must have transverse field components to satisfy Maxwell's equations, leading to divergence. We write a script that numerically calculates the flow of energy, described by the Poynting vector, to prove that the propagation is almost entirely along z , with a very small but non-zero transverse component responsible for the beam's spread.

9.2 Background Theory

9.2.1 The Poynting Vector

The Poynting vector, $\mathbf{S}$, describes the directional energy flux (the rate of energy transfer per unit area) of an electromagnetic field. For time-harmonic fields, the time-averaged Poynting vector is given by:

$$\mathbf{S} = \frac{1}{2} \text{Re}(\mathbf{E} \times \mathbf{H}^*)$$

where $\mathbf{E}$ is the complex electric field vector, $\mathbf{H}^*$ is the complex conjugate of the magnetic field vector, and $\text{Re}(\cdot)$ denotes the real part. The direction of $\mathbf{S}$ at any point is the direction of energy flow.

9.2.2 Paraxial Approximation

For a Gaussian beam propagating along z , the paraxial approximation assumes the field is primarily transverse (e.g., $E_x \gg E_z, E_y$) and that its properties change slowly along z . We test the paraxial assumption by calculating all components of $\mathbf{S}$ and showing that $S_z \gg S_x, S_y$ in the new script.

9.3 Mathematical Formulation

9.3.1 From Maxwell's Equations to the H-Field

We need both $\mathbf{E}$ and $\mathbf{H}$ to find $\mathbf{S}$. We start by assuming the scalar field from our simulation is the dominant x -polarized component, so $\mathbf{E} \approx [E_x, 0, 0]$. From Faraday's Law:

$$\nabla \times \mathbf{E} = -i\omega\mu_0\mathbf{H}$$

Assuming a non-magnetic medium where $\mu_r = 1$ and using $\omega = ck$ and $c = 1/\sqrt{\epsilon_0\mu_0}$, we can write $\omega\mu_0 = k$ (assuming $c = 1, \mu_0 = 1$ for simplicity, as in the code). This gives:

$$\mathbf{H} = \frac{1}{-ik}(\nabla \times \mathbf{E})$$

Calculating the curl of $\mathbf{E} = [E_x, 0, 0]$:

$$\nabla \times \mathbf{E} = \left(\frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z} \right) \hat{x} + \left(\frac{\partial E_x}{\partial z} - \frac{\partial E_z}{\partial x} \right) \hat{y} + \left(\frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} \right) \hat{z}$$

Since we assume $E_y \approx 0$ and $E_z \approx 0$, this simplifies significantly to:

$$\nabla \times \mathbf{E} \approx \left(\frac{\partial E_x}{\partial z} \right) \hat{y} - \left(\frac{\partial E_x}{\partial y} \right) \hat{z}$$

Therefore, the magnetic field components are:

$$H_x \approx 0 \quad | \quad H_y \approx \frac{1}{-ik} \frac{\partial E_x}{\partial z} \quad | \quad H_z \approx \frac{1}{-ik} \left(-\frac{\partial E_x}{\partial y} \right) = \frac{i}{k} \frac{\partial E_x}{\partial y}$$

9.3.2 The Poynting Vector Calculation

Now we compute the cross product $\mathbf{E} \times \mathbf{H}^* = [E_x, 0, 0] \times [0, H_y^*, H_z^*]$:

$$\mathbf{E} \times \mathbf{H}^* = \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ E_x & 0 & 0 \\ 0 & H_y^* & H_z^* \end{vmatrix} = (0 \cdot H_z^* - 0 \cdot H_y^*)\hat{x} - (E_x H_z^* - 0)\hat{y} + (E_x H_y^* - 0)\hat{z}$$

$$\mathbf{E} \times \mathbf{H}^* = 0\hat{x} - (E_x H_z^*)\hat{y} + (E_x H_y^*)\hat{z}$$

Finally, the time-averaged Poynting vector components are:

$$S_x = 0 \quad | \quad S_y = \frac{1}{2} \text{Re}(-E_x H_z^*) \quad | \quad S_z = \frac{1}{2} \text{Re}(E_x H_y^*)$$

9.3.3 Numerical Derivatives

The code must compute $\frac{\partial E_x}{\partial y}$ and $\frac{\partial E_x}{\partial z}$ numerically.

- **Transverse** ($\frac{\partial E_x}{\partial y}$): This is computed from the 2D grid $E_x(x, y)$ using a standard central difference, conveniently provided by MATLAB's `gradient` function.
- **Longitudinal** ($\frac{\partial E_x}{\partial z}$): We use the basic definition of a derivative to approximate this. We use the central difference formula, after running the simulation three times: at z , $z + \Delta z$, and $z - \Delta z$:

$$\frac{\partial E_x}{\partial z} \approx \frac{E_x(z + \Delta z) - E_x(z - \Delta z)}{2\Delta z}$$

9.4 Plots and Results

- **Figure 6: Component Magnitudes (Subplots)**

- **S_z Plot:** This plot shows the longitudinal power flow. It will look identical to the beam's intensity profile, as S_z is the dominant component and is proportional to $|E_x|^2$ (since $H_y \propto E_x$).
- **S_y Plot:** This plot shows the transverse power flow. The pattern will be much weaker (note the color bar scale) and will likely show positive values on one side of the y-axis and negative on the other, indicating energy flowing "outward" from the beam center, which is the essence of divergence.

- **Console Output:** The script prints the ratio of the maximum S_z to the maximum S_y . This value will be very large (e.g., 10^2 to 10^4), providing the numerical proof that the paraxial assumption is valid and that energy flows (approximately) along z .

- **Figure 7: Quiver Plot**

- This plot shows the main S_z intensity as a background color.
- The red arrows (quivers) show the *direction* of the transverse power flow (S_x, S_y). For our E_x polarization, $S_x = 0$, so the arrows will only point along $\pm y$. They will point away from the x -axis ($y = 0$), visually confirming the outward flow of energy.

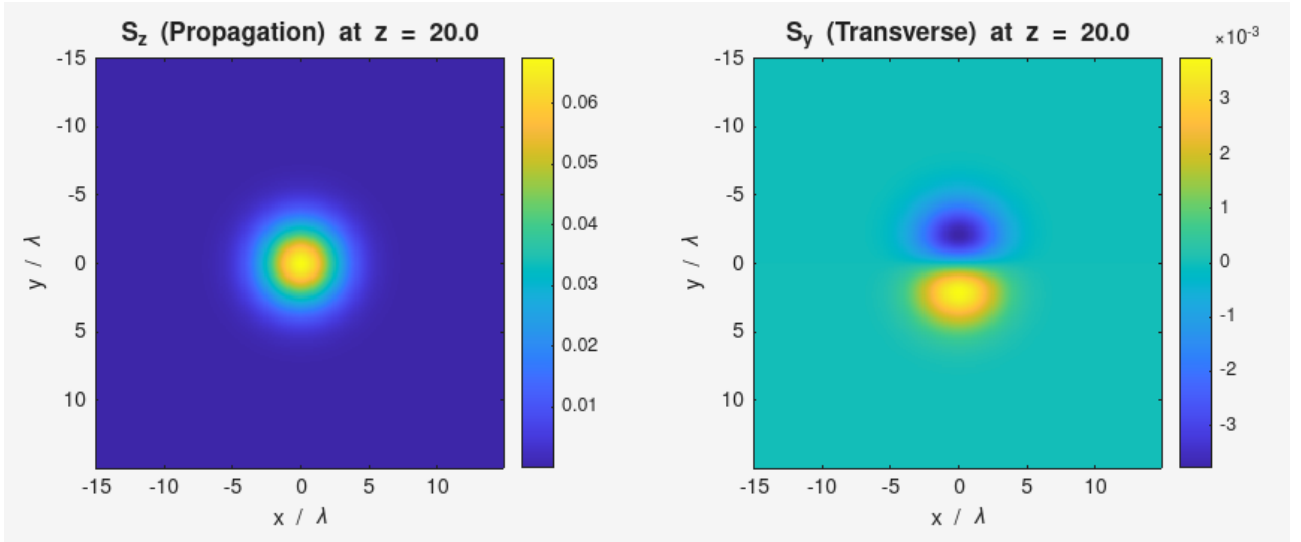


Figure 6: Comparison of Poynting vector components. The plot for S_z (left) will show the familiar Gaussian intensity profile. The plot for S_y (right) will show a much weaker, more complex pattern, indicating transverse energy flow.

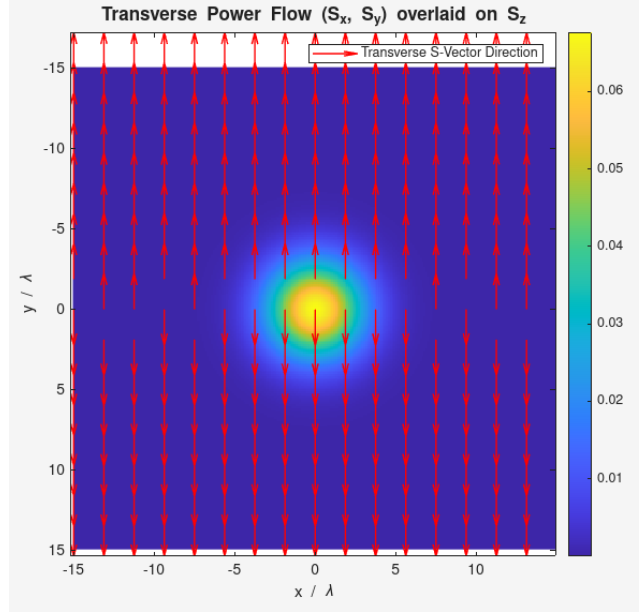


Figure 7: Direction of the transverse power flow

10 Plotting the beam at an interface

10.1 Introduction

We write a script to simulate a focused beam hitting a boundary between two different materials at an angle. It utilizes the Angular Spectrum Method (ASM) to handle the complex propagation and the Fresnel equations to account for the interaction at the interface. The code can accurately predict reflection, refraction (the bending of light), and more advanced phenomena, such as Total Internal Reflection (TIR) and Brewster's angle, simply by adjusting the input parameters.

10.2 Mathematical Formulation

10.2.1 Decomposing the Tilted Beam

1. **Initial Field:** $E(x, y, 0) = \exp(-(x^2 + y^2)/w_0^2)$.

2. **Phase Tilt:** To make the beam incident at an angle θ_i in the x-z plane, we apply a linear phase ramp:

$$E_{\text{tilted}}(x, y, 0) = E(x, y, 0) \cdot e^{-ik_1 \sin(\theta_i)x}$$

This $k_{x,\text{inc}} = k_1 \sin(\theta_i)$ is the central k_x component of the beam.

3. **Angular Spectrum:** The initial spectrum is the Fourier transform of this tilted field:

$$A_0(k_x, k_y) = \mathcal{F}\{E_{\text{tilted}}(x, y, 0)\}$$

10.2.2 Fresnel Equations

For each plane wave (k_x, k_y) , we must calculate the reflection $R(k_x, k_y)$ and transmission $T(k_x, k_y)$ coefficients. This requires the angle of incidence for *that specific plane wave*. We find these from the k_z components:

$$\cos(\theta_1) = \frac{k_{z1}}{k_1} \quad | \quad \cos(\theta_2) = \frac{k_{z2}}{k_2}$$

The Fresnel equations are then:

$$R_s = \frac{n_1 \cos(\theta_1) - n_2 \cos(\theta_2)}{n_1 \cos(\theta_1) + n_2 \cos(\theta_2)} \quad T_s = \frac{2n_1 \cos(\theta_1)}{n_1 \cos(\theta_1) + n_2 \cos(\theta_2)}$$

$$R_p = \frac{n_2 \cos(\theta_1) - n_1 \cos(\theta_2)}{n_2 \cos(\theta_1) + n_1 \cos(\theta_2)} \quad T_p = \frac{2n_1 \cos(\theta_1)}{n_2 \cos(\theta_1) + n_1 \cos(\theta_2)}$$

10.2.3 Superposition of Fields

The code calculates the total field at z_{obs} based on its location.

- **Case 1:** $z_{obs} < z_{interface}$ (**In Medium 1**) The total field is the sum of the forward-going incident beam and the backward-going reflected beam.

$$A_{inc}(z_{obs}) = A_0 \cdot e^{-ik_{z1}z_{obs}}$$

$$A_{refl}(z_{obs}) = \left((A_0 \cdot e^{-ik_{z1}z_{iface}}) \cdot R \right) \cdot e^{+ik_{z1}(z_{iface}-z_{obs})}$$

$$A_{final} = A_{inc}(z_{obs}) + A_{refl}(z_{obs})$$

- **Case 2:** $z_{obs} > z_{interface}$ (**In Medium 2**) The total field is just the transmitted beam, which has been propagated to the interface in medium 1, multiplied by T , and then propagated from the interface in medium 2.

$$A_{trans_at_iface} = (A_0 \cdot e^{-ik_{z1}z_{iface}}) \cdot T$$

$$A_{final} = A_{trans_at_iface} \cdot e^{-ik_{z2}(z_{obs}-z_{iface})}$$

Finally, the real-space field is reconstructed: $E_{final}(x, y) = \mathcal{F}^{-1}\{A_{final}\}$.

10.3 Plots

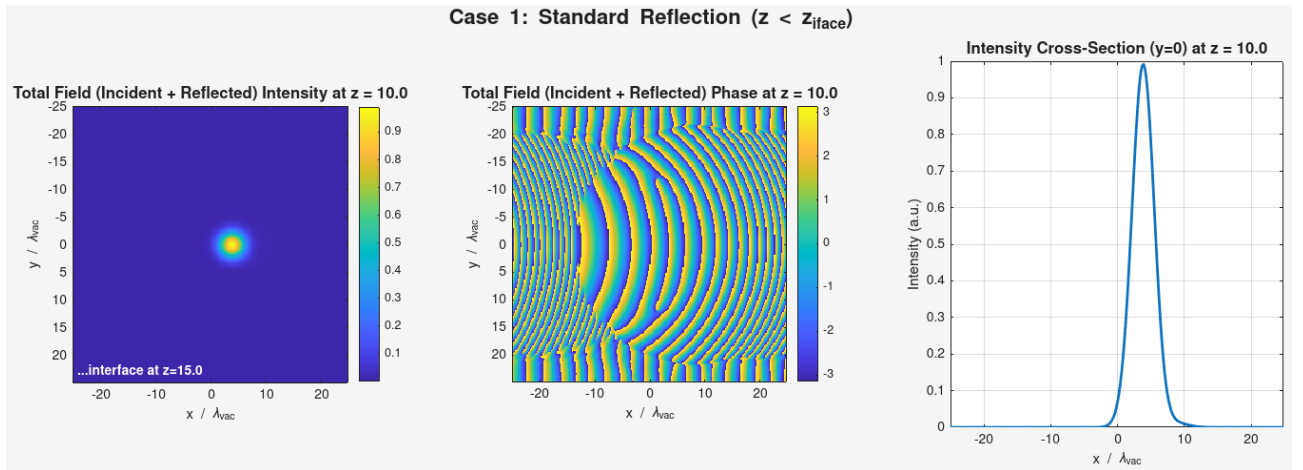


Figure 8: Case 1: Standard Reflection (s-pol, $\theta_i = 20^\circ$)

In all cases, the plots show severe, unphysical aberrations like spatial chirps and phase ripples, which should ideally not be observed. A possible mistake is to model a tilted Gaussian beam as a simple product of a circular Gaussian amplitude profile and a linear phase ramp.

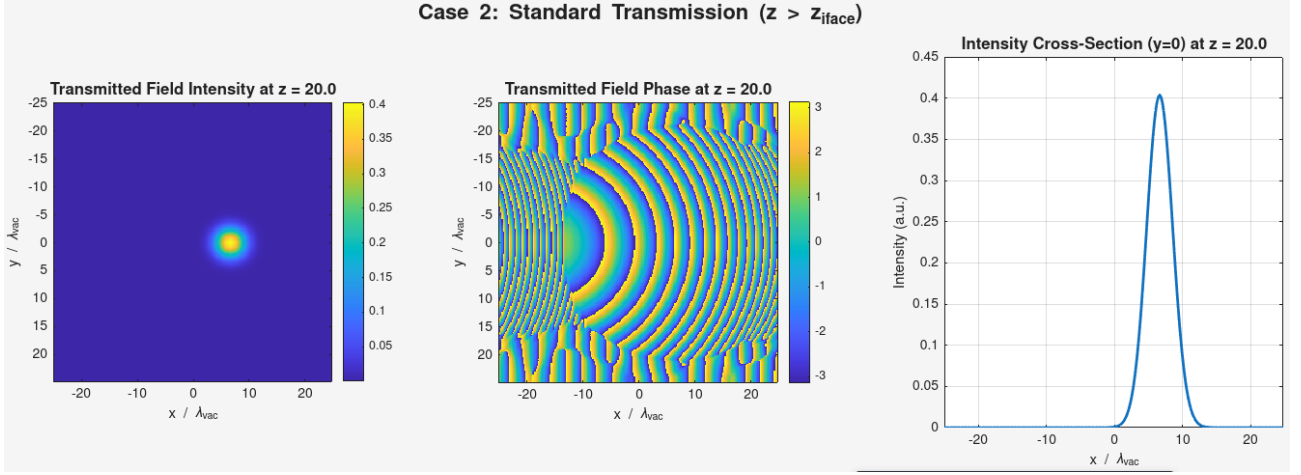


Figure 9: Case 2: Standard Transmission (s-pol, $\theta_i = 20^\circ$)

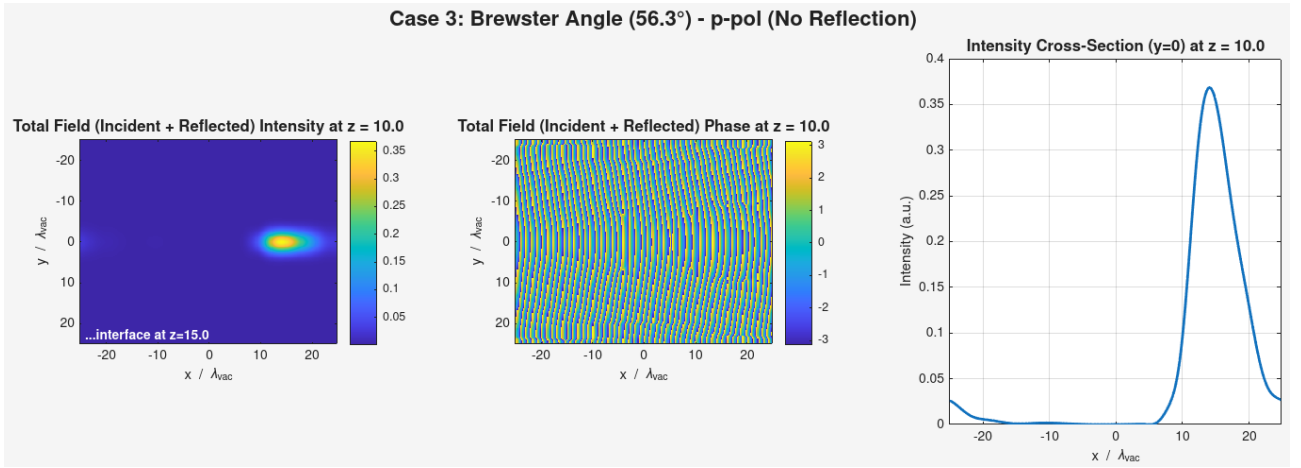


Figure 10: Case 3: Brewster's Angle (p-pol, No Reflection)

10.4 The Current Model (Circular Projection)

10.4.1 Mathematical Formulation

The current model defines the input field at the $z = 0$ plane as the product of two independent functions:

1. **A circular Gaussian amplitude:** $E_{\text{amp}}(x, y) = \exp\left(-\frac{x^2 + y^2}{w_0^2}\right)$
2. **A linear phase ramp:** $E_{\text{phase}}(x) = \exp(-ik_1 \sin(\theta_i)x)$

The resulting input field is:

$$E_{\text{original}}(x, y, 0) = \exp\left(-\frac{x^2 + y^2}{w_0^2}\right) \cdot \exp(-ik_1 \sin(\theta_i)x)$$

10.4.2 Physical Interpretation

This field is **not** a tilted Gaussian beam. Mathematically, it describes the field immediately after a standard, non-tilted, z -propagating Gaussian beam (with a flat phase front at $z = 0$) passes through an ideal, thin prism.

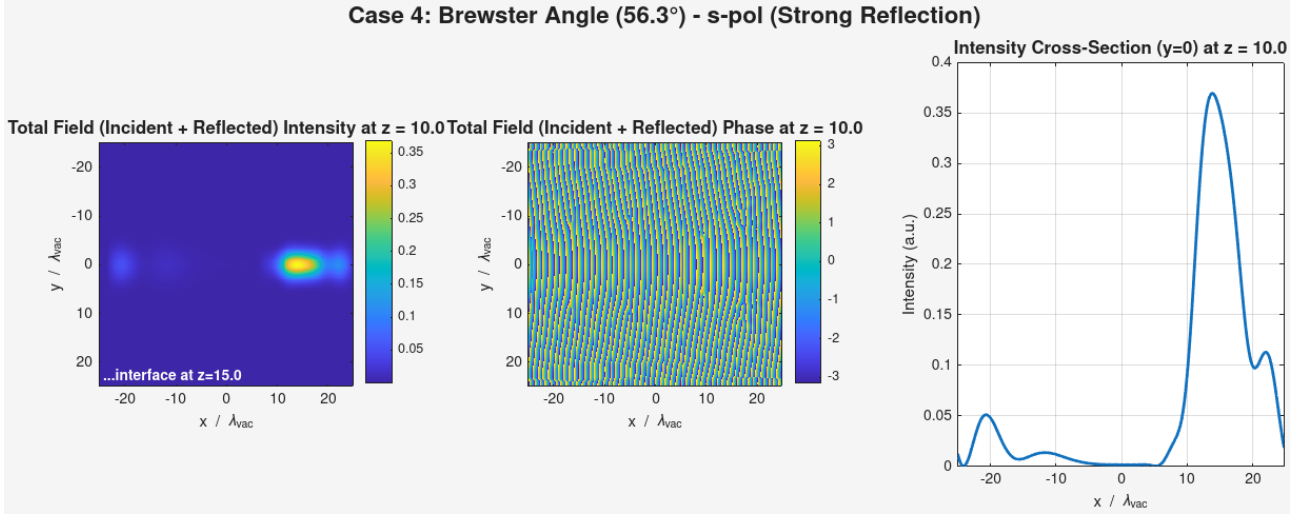


Figure 11: Case 4: Brewster's Angle (s-pol, Strong Reflection)

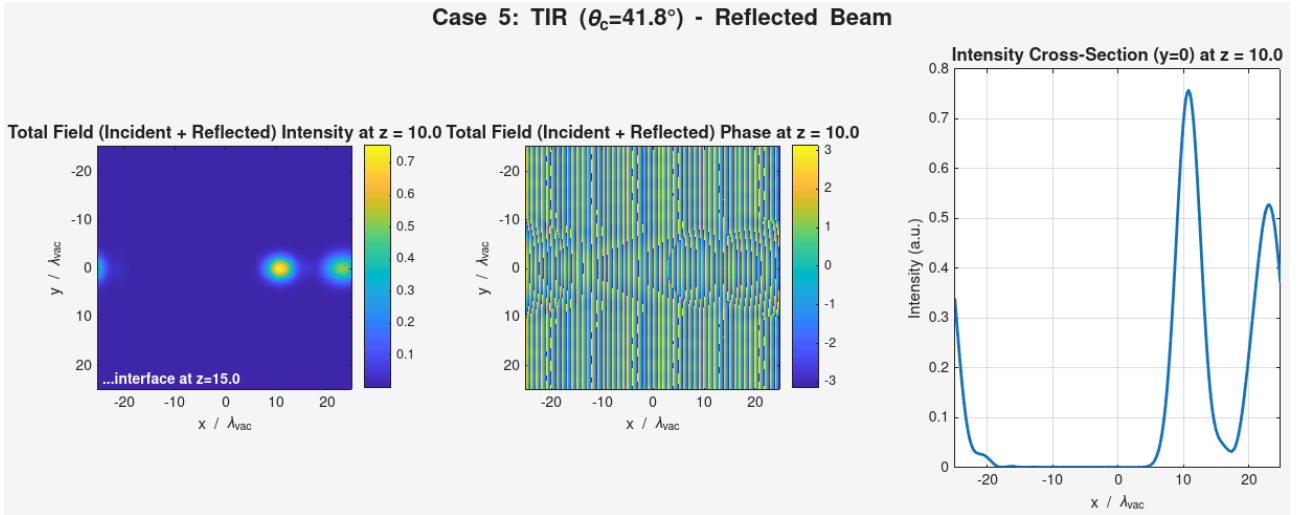


Figure 12: Case 5: Total Internal Reflection (TIR) - Reflected Beam

- The $\exp(-\frac{x^2+y^2}{w_0^2})$ term describes the standard z -propagating beam.
- The $\exp(-ik_1 \sin(\theta_i)x)$ term is the exact transfer function of a phase wedge (prism) that kicks the beam by an angle θ_i .

This prism-kicked beam is now highly **aberrated**. Its amplitude profile (a circle) and its phase fronts (sheared) are mismatched. The ASM, being a correct propagator, propagates this aberrated field. The spatial chirp and phase rippling observed are not errors in the ASM; they are the *correct physical consequences* of propagating this unphysical, aberrated input field.

10.5 The Corrected Model (Elliptical Projection)

10.5.1 Mathematical Derivation: The Rotated Beam Model

To find the correct input field at $z = 0$, we must find the footprint of a true, pure Gaussian beam that is propagating at an angle θ_i relative to the z -axis.

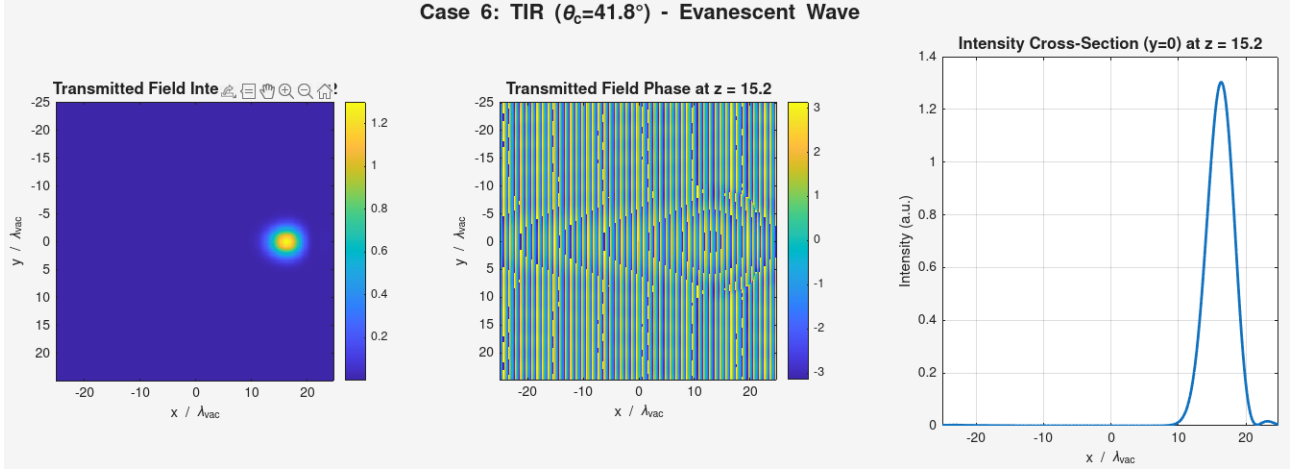


Figure 13: Case 6: TIR - Evanescent Wave

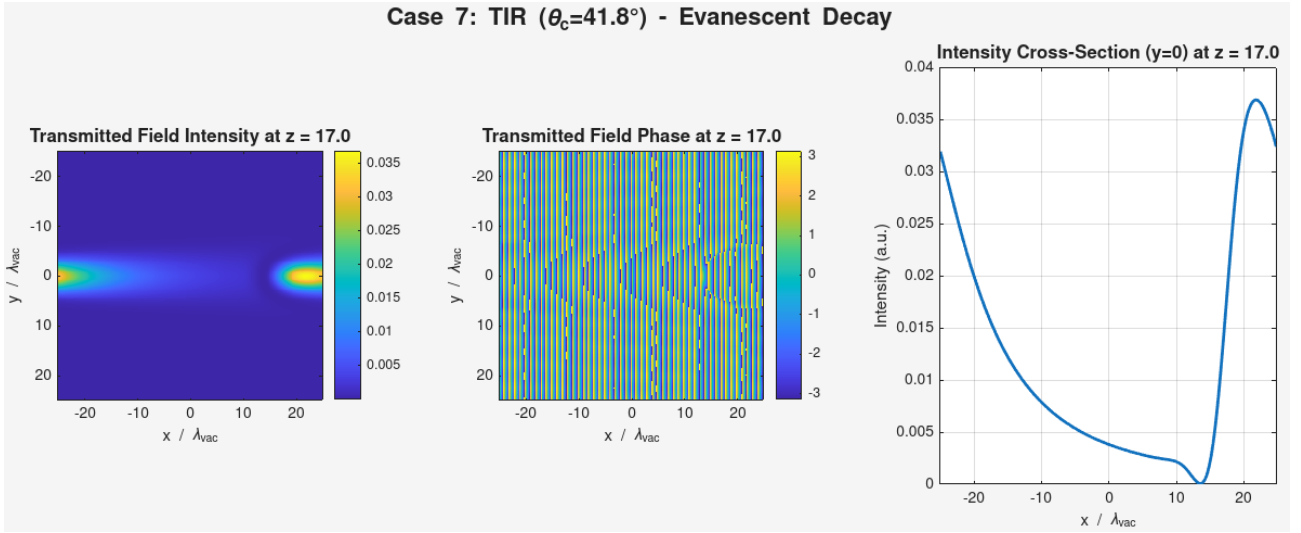


Figure 14: Case 7: TIR - Evanescent Decay

1. **Define two coordinate frames:**

- **Lab Frame (x, y, z):** The simulation frame. The interface is at $z = z_{\text{iface}}$.
- **Beam Frame (x', y', z'):** The beam's "natural" frame, where it propagates along the z' -axis.

2. **Relate the frames via rotation:** The beam frame is rotated by θ_i around the y -axis. The transformation for any point (x, y, z) is:

$$\begin{aligned} x' &= x \cos(\theta_i) - z \sin(\theta_i) \\ y' &= y \\ z' &= z \cos(\theta_i) + x \sin(\theta_i) \end{aligned}$$

3. **Define the field in the Beam Frame:** In its own frame, the beam is a pure, z' -propagating Gaussian with its waist w_0 at the origin $(x', y', z') = (0, 0, 0)$. The field is given by its amplitude

and propagation phase:

$$E'(x', y', z') \approx \exp\left(-\frac{x'^2 + y'^2}{w_0^2}\right) \cdot \exp(-ik_1 z')$$

4. **Find the field in the Lab Frame at $z = 0$:** We need to find $E(x, y, 0)$. We do this by setting $z = 0$ in the transformation equations:

- $x' = x \cos(\theta_i)$
- $y' = y$
- $z' = x \sin(\theta_i)$

Now, substitute these into the E' equation:

$$E_{\text{correct}}(x, y, 0) = \exp\left(-\frac{(x \cos(\theta_i))^2 + y^2}{w_0^2}\right) \cdot \exp(-ik_1(x \sin(\theta_i)))$$

10.5.2 Physical Interpretation

Let's analyze this correct field:

$$E_{\text{correct}}(x, y, 0) = \underbrace{\exp\left(-\frac{x^2}{(w_0/\cos\theta_i)^2} - \frac{y^2}{w_0^2}\right)}_{\text{Elliptical Amplitude}} \cdot \underbrace{\exp(-ik_1 \sin(\theta_i)x)}_{\text{Phase Tilt}}$$

This equation is profoundly different from the previous one.

- **Phase:** The phase term $\exp(-ik_1 \sin(\theta_i)x)$ is identical. This is the correct "aim" of the beam.
- **Amplitude:** The amplitude term is **elliptical**, not circular.
 - The waist in y is $w_{0y} = w_0$.
 - The waist in x is $w_{0x} = w_0 / \cos(\theta_i)$.

Since $\cos(\theta_i) \leq 1$, the projected waist w_{0x} is *always larger* than w_0 . This is the physically correct footprint of a tilted flashlight beam on a wall.

10.6 Comparative Analysis of Simulation Results

New plots have a dark theme in order to differentiate them from the previous plots.

We selectively analyze 2 cases: Brewster Angle (s-pol) - Reflected Beam, and Total Internal Reflection (TIR) - Evanescent Decay.

10.6.1 Case 4: Brewster Angle (s-pol) - Reflected Beam

At the Brewster angle, s-polarized light experiences strong reflection. The reflected field should be a clean, single Gaussian beam.

- **Erroneous Model:** The reflected beam is distorted. The cross-section plot shows a significant, unphysical side-lobe around $x = -15$. This is a direct result of the aberrated prism-kicked field reflecting and interfering with itself.
- **Corrected Model (Right):** The reflected beam is a single, clean Gaussian spot, exactly as expected. The cross-section shows a single, dominant peak.

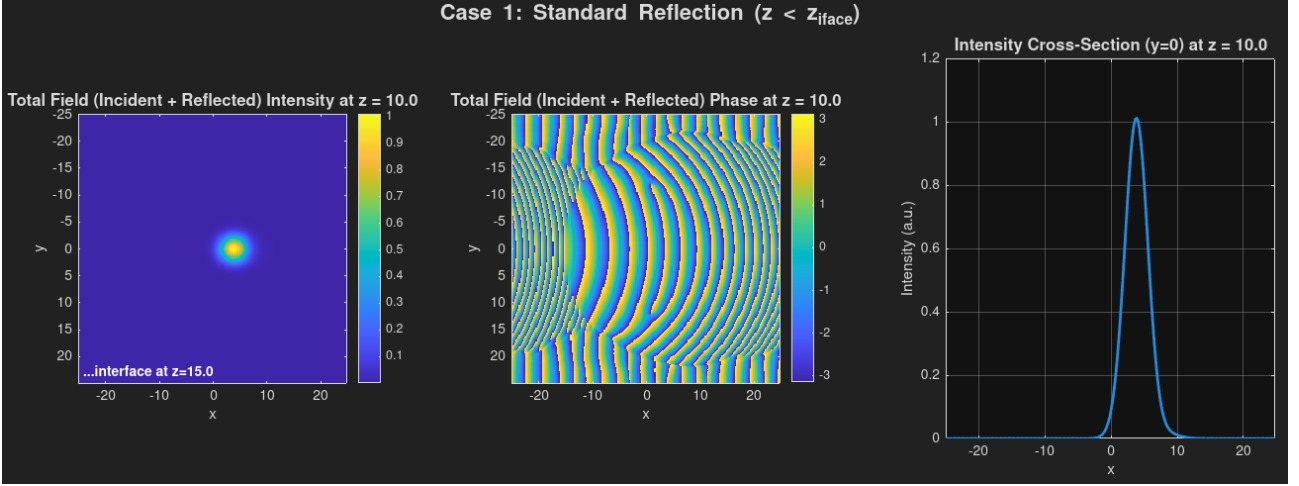


Figure 15: Corrected Case 1: Standard Reflection (s-pol, $\theta_i = 20^\circ$)

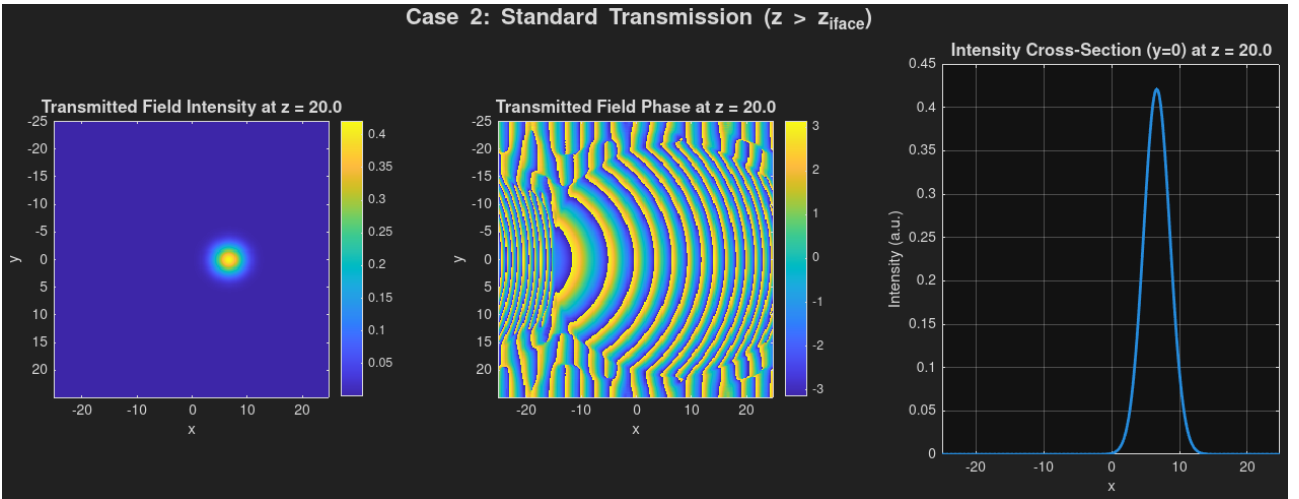


Figure 16: Corrected Case 2: Standard Transmission (s-pol, $\theta_i = 20^\circ$)

10.7 Case 7: Total Internal Reflection (TIR) - Evanescent Decay

In the second medium (beyond the interface) during TIR, we expect a non-propagating evanescent field that decays exponentially away from the interface.

- **Erroneous Model:** The evanescent field shows strange, non-physical beading and distortion. The cross-section shows a non-monotonic, complex shape, not a simple decay. The peak intensity is also suspiciously high (~ 0.035).
- **Corrected Model (Right):** The evanescent field is a smooth, elongated spot, as expected. The cross-section shows a smooth, decaying profile. The peak intensity is also much lower (~ 0.007), indicating the prism model was unphysically coupling far too much energy into aberrated evanescent modes.

We could do a more rigorous analysis of the error by first mathematically deriving the fields for the incorrect code, matching them to all the obtained plots, follows by repeating the same for the corrected code. I am still not sure if the new code is giving the correct so definitely a better mathematical analysis is required.

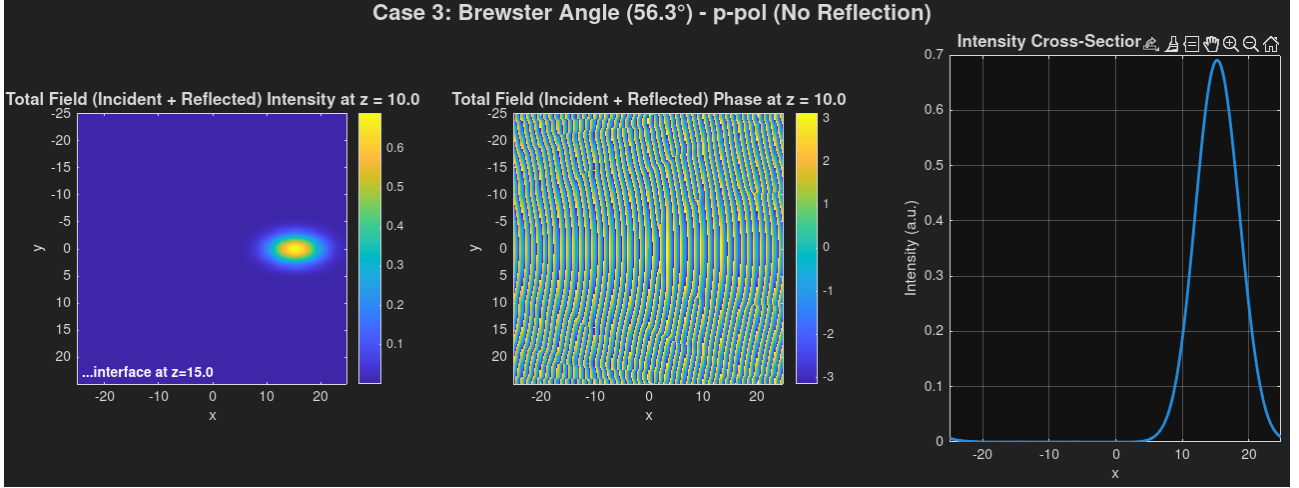


Figure 17: Corrected Case 3: Brewster's Angle (p-pol, No Reflection)

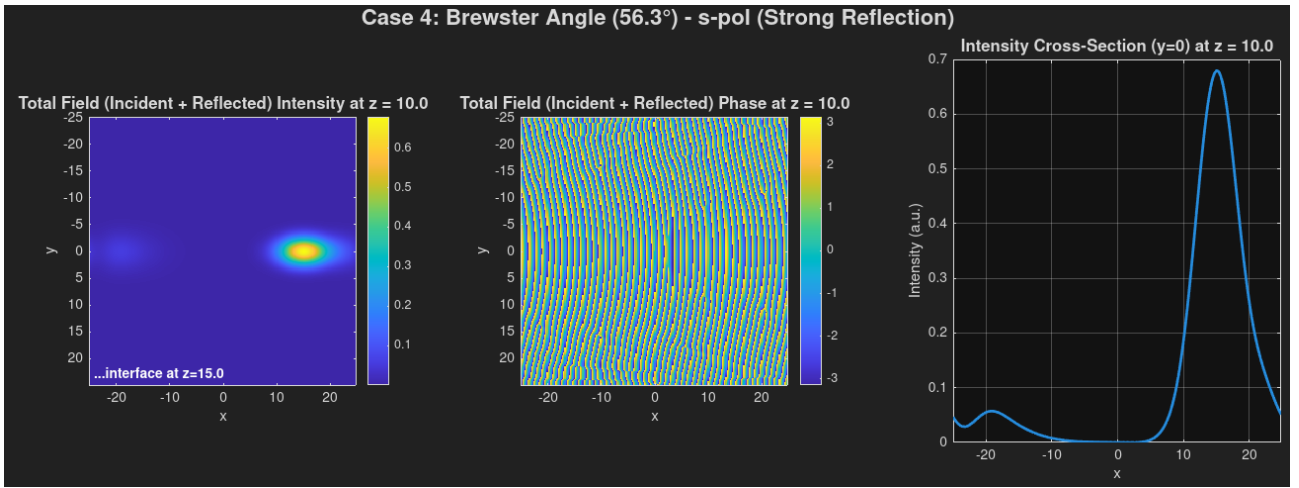


Figure 18: Corrected Case 4: Brewster's Angle (s-pol, Strong Reflection)

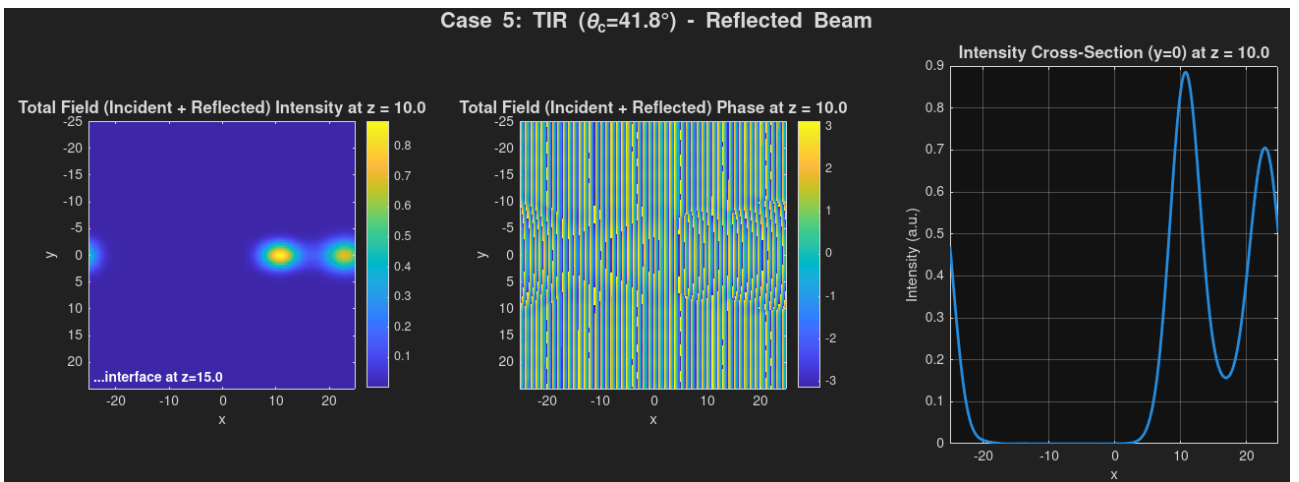


Figure 19: Corrected Case 5: Total Internal Reflection (TIR) - Reflected Beam

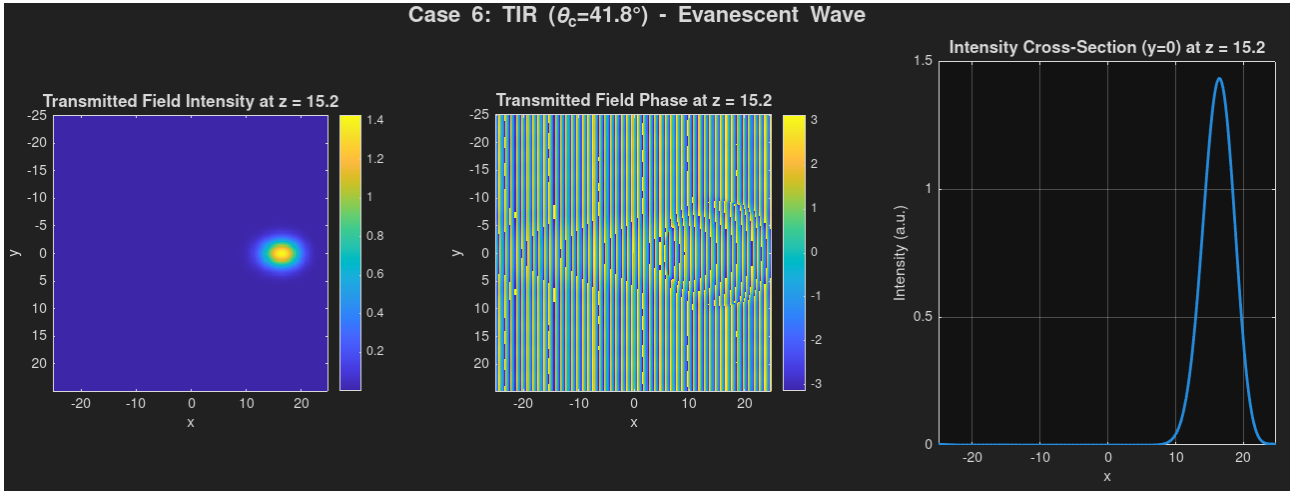


Figure 20: Corrected Case 6: TIR - Evanescent Wave

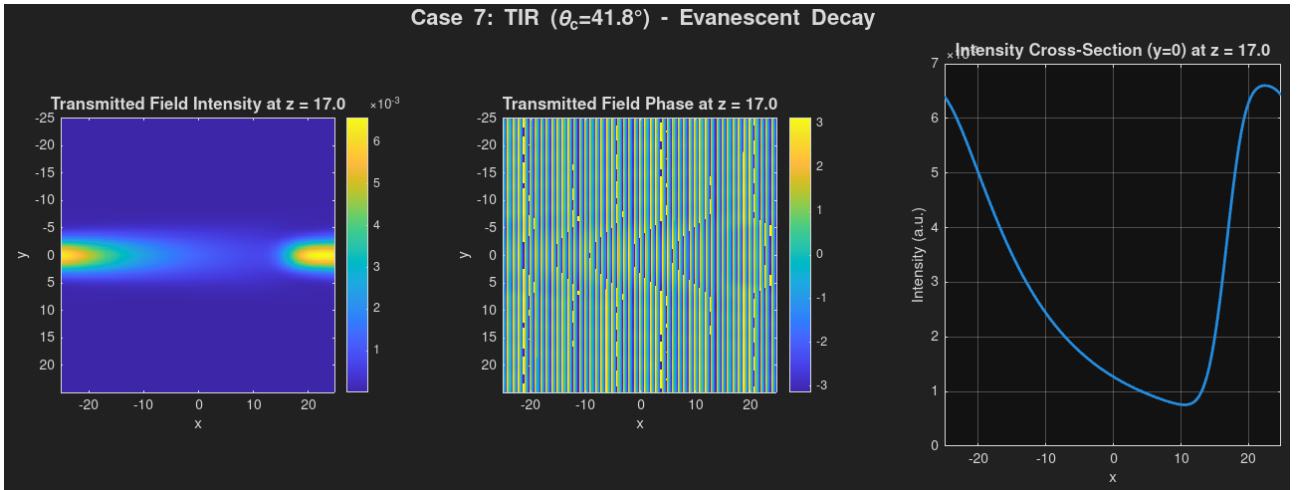


Figure 21: Corrected Case 7: TIR - Evanescent Decay